

Inheritance and Abstraction



Christian Wenz

Consultant, Software Architect

@chwenz



Understanding Inheritance

Class: Event

Properties:

- date
- venue
- artist

Methods:

- cancel()
- reschedule()

Class: Concert

Properties:

- artist

Class: Tour

Methods:

- reschedule()



Inheritance

event.class.ts

```
class Event {  
  date: Date;  
  venue: string;  
}
```

concert.class.ts

```
class Concert extends Event {  
  artist: string;  
}  
  
class Tour extends Event {  
  reschedule(newDate: Date): boolean {  
    // ...  
  }  
}
```



Using super()

event.class.ts

```
class Event {  
  constructor() {  
    console.log('Event created.');  }  
}
```

concert.class.ts

```
class Concert extends Event {  
  constructor() {  
    super();  
  }  
}
```



Overriding Methods

event.class.ts

```
class Event {  
  toDisplayName(): string {  
    return this.title;  
  }  
}
```

concert.class.ts

```
class Concert extends Event {  
  toDisplayName(): string {  
    if (this.artist === '') {  
      return super.toDisplayName();  
    } else {  
      return this.artist + ' @ ' +  
        this.venue;  
    }  
  }  
}
```



Access Modifiers

public

private

protected

readonly



```
enum EventType {  
    Concert,  
    Tour  
}  
  
class Concert extends Event {  
    static readonly type: EventType = EventType.Concert;  
}
```

Static Class Members

Properties independent of any specific class instance



Interfaces

ievent.class.ts

```
interface IEvent {  
    cancel(): boolean;  
}
```

concert.class.ts

```
class Concert implements IEvent {  
    cancel(): boolean {  
        // ...  
    }  
}
```



Abstract Classes

event.class.ts

```
abstract class Event {  
  abstract cancel(): boolean;  
  
  toDisplayName(): string {  
    return this.title;  
  }  
}
```

concert.class.ts

```
class Concert extends Event {  
  cancel(): boolean {  
    // ...  
  }  
}
```

